Lesson 4

Support Vector Machines & Kernels

> *"When solving a problem of interest, do not solve a more general problem as an intermediate step. Try to get the answer that you really need but not a more general one."*
>
> — Vladimir Vapnik, *Statistical Learning Theory* (1998)

Introduction & Motivation

After regression, classification, and latent-variable compression, we now ask a more geometric question: if two physical regimes can be separated, which separating boundary is safest? In chemical engineering, the answer is rarely "the boundary that classifies the training data best." The operational answer is usually "the boundary with the largest safety buffer."

Support Vector Machines (SVMs) formalize this idea. They replace the vague notion of a good classifier with a strict margin-maximization problem. This makes SVMs especially valuable in High-Dimension, Low-Sample-Size regimes, where failures are rare, data is expensive, and a model must remain stable even when the number of measured features exceeds the number of examples.

Lesson Objectives:

- Derive the hard-margin SVM from the geometry of distance to a hyperplane.
- Use Lagrange multipliers and KKT conditions to obtain the dual optimization problem.
- Understand why only support vectors determine the final decision boundary.
- Introduce kernel functions as implicit high-dimensional feature maps.
- Apply the SVM framework to rare-event detection in battery manufacturing.

Q&A: Why margins instead of accuracy?
Question: If two classifiers both separate the training data perfectly, why should we prefer the one with the wider margin?
Answer: In a physical process, future samples will not land exactly where the historical samples landed. Sensor drift, thermal noise, calibration error, and batch-to-batch variability move points around the state space. A wide margin creates a buffer against these unavoidable disturbances.

The Physical Reality: The Boundary Is a Safety Argument

In classification problems like runaway detection or phase-boundary monitoring, the SVM hyperplane is not merely a mathematical separator. It becomes an operational safety argument: how far is the current process state from the nearest historical failure state?

4.1 The Maximum Margin Principle

In Lesson 2, we utilized Logistic Regression to draw a decision boundary between thermodynamic phases. However, Logistic Regression merely finds *a* boundary that separates the classes. If the classes are perfectly separable, there are infinitely many valid linear boundaries. Which one is best?

In safety-critical physical systems, we do not just want to separate failures from nominal states; we want to separate them with the maximum possible physical buffer. This is the foundational intuition behind the **Support Vector Machine (SVM)**.

Assume we have a binary classification dataset $\mathcal{D}$ consisting of N samples. Let $x_i \in \mathbb{R}^p$ be our sensor readings and $y_i \in \{-1, 1\}$ be the class labels (e.g., -1 for nominal, 1 for failure). We define a linear hyperplane as:

$$w^T x + b = 0 \tag{4.1}$$

where $w \in \mathbb{R}^p$ is the normal vector to the hyperplane, and $b \in \mathbb{R}$ is the scalar bias.

If the data is linearly separable, we can scale w and b such that the nearest data points to the hyperplane satisfy exactly $w^T x_i + b = 1$ (for

$y_i = 1$) and $w^T x_i + b = -1$ (for $y_i = -1$). We can elegantly combine these into a single mathematical constraint:

$$y_i(w^T x_i + b) \geq 1 \quad \forall i \in \{1, \ldots, N\} \tag{4.2}$$

The geometric distance between these two margin boundaries is exactly $\frac{2}{\|w\|}$. To maximize this safety margin, we must minimize $\|w\|$. For mathematical convenience, we minimize the squared L_2-norm, $\frac{1}{2}\|w\|^2$.

Definition: The Primal SVM Optimization

The hard-margin SVM is formulated as a constrained convex optimization problem:

$$\begin{aligned} \underset{w,b}{\text{minimize}} \quad & \frac{1}{2} w^T w \\ \text{subject to} \quad & y_i(w^T x_i + b) \geq 1 \quad \forall i \end{aligned} \tag{4.3}$$

4.2 The KKT Conditions and the Dual Problem

To solve the constrained optimization in Equation 4.3, we introduce Lagrange multipliers $\alpha_i \geq 0$ for each of the N constraints. The Lagrangian function is:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} w^T w - \sum_{i=1}^{N} \alpha_i \Big[y_i(w^T x_i + b) - 1 \Big] \tag{4.4}$$

Because this is a convex optimization problem with affine constraints, the optimal solution must strictly satisfy the **Karush-Kuhn-Tucker (KKT) conditions**. These five conditions govern the entire mathematical foundation of the SVM:

1. **Stationarity (w):** The gradient of the Lagrangian with respect to w must be zero:

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^{N} \alpha_i y_i x_i = 0 \implies w = \sum_{i=1}^{N} \alpha_i y_i x_i \tag{4.5}$$

2. **Stationarity (b):** The derivative with respect to b must be zero:

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{N} \alpha_i y_i = 0 \implies \sum_{i=1}^{N} \alpha_i y_i = 0 \tag{4.6}$$

3. **Primal Feasibility:** The original margin constraints must hold:

$$y_i(w^T x_i + b) - 1 \geq 0 \quad \forall i \tag{4.7}$$

4. **Dual Feasibility:** The Lagrange multipliers must be non-negative:

$$\alpha_i \geq 0 \quad \forall i \tag{4.8}$$

5. **Complementary Slackness:** The product of the multiplier and the constraint must be zero:

$$\alpha_i \Big[y_i(w^T x_i + b) - 1 \Big] = 0 \quad \forall i \tag{4.9}$$

Complementary slackness (Condition 5) dictates a profound physical reality: if a data point x_i lies safely behind the margin ($y_i(w^T x_i + b) > 1$), then its multiplier α_i **must be exactly zero**. The only points with $\alpha_i > 0$ are those resting exactly on the margin boundaries. These specific points are the **Support Vectors**.

By substituting Equations 4.5 and 4.6 back into the Lagrangian (Equation 4.4), we eliminate the primal variables w and b, yielding the **Dual Formulation**:

Definition: The Dual SVM Optimization

$$\begin{aligned} \underset{\alpha}{\text{maximize}} \quad & \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j (x_i^T x_j) \\ \text{subject to} \quad & \alpha_i \geq 0 \quad \forall i, \quad \sum_{i=1}^{N} \alpha_i y_i = 0 \end{aligned} \tag{4.10}$$

4.3 Solving the SVM in Practice

How do we actually find the optimal boundary? We do not solve for w directly. Instead, we solve the Dual Problem (Equation 4.10) to find the optimal vector of multipliers, α^*.

Because the dual is a strictly convex Quadratic Programming (QP) problem, it is solved using highly efficient numerical algorithms like **Sequential Minimal Optimization (SMO)**.

Once the QP solver outputs the optimal α^*, we construct the physical boundary as follows:

1. **Find the weights:** Calculate w using Equation 4.5. Notice that because most $\alpha_i = 0$, w is defined *only* by the handful of Support Vectors.
2. **Find the bias:** Once w is known, the only remaining unknown is the intercept b, which shifts the hyperplane without changing its orientation. To recover it, pick any single Support Vector x_k (where $\alpha_k > 0$). A Support Vector is not merely close to the boundary; by definition it lies exactly on one of the two margin planes. Therefore its constraint is active:

$$y_k(w^T x_k + b) = 1 \tag{4.11}$$

Since $y_k \in \{-1, 1\}$, we have $y_k^2 = 1$. Multiplying both sides by y_k gives:

$$\begin{aligned} y_k^2(w^T x_k + b) &= y_k \\ w^T x_k + b &= y_k \\ b &= y_k - w^T x_k \end{aligned} \tag{4.12}$$

In practice, several Support Vectors may be available. To reduce numerical noise, implementations often compute b from each Support Vector and average the results:

$$b = \frac{1}{|\mathcal{S}|} \sum_{k \in \mathcal{S}} \left(y_k - w^T x_k \right) \tag{4.13}$$

where $\mathcal{S}$ is the set of Support Vectors. Physically, this means we use the points that sit exactly on the safety margin to anchor the final location of the decision boundary.

4.4 The Kernel Trick

Look closely at the Dual Formulation (Equation 4.10). The physical input vectors x_i only ever appear as a *dot product* $(x_i^T x_j)$.

If our physical data is not linearly separable, we can mathematically project it into a higher-dimensional space using a transformation function

$\phi(x)$. The dot product becomes $\phi(x_i)^T\phi(x_j)$. However, computing $\phi(x)$ for an infinite-dimensional space is computationally impossible.

The **Kernel Trick** allows us to bypass computing $\phi(x)$ entirely. We replace the dot product with a Kernel function $K(x_i, x_j)$ that calculates the inner product in the high-dimensional space directly from the original low-dimensional inputs:

$$K(x_i, x_j) = \phi(x_i)^T\phi(x_j) \tag{4.14}$$

The standard choice in engineering is the **Radial Basis Function (RBF) Kernel**, which mathematically projects the data into an infinite-dimensional space:

$$K(x_i, x_j) = \exp\left(-\gamma\|x_i - x_j\|^2\right) \tag{4.15}$$

where $\gamma > 0$ dictates the radius of influence of a single support vector. Using the RBF kernel allows the SVM to draw highly complex, non-linear boundaries around physical failure states without ever leaving the dual space.

Figure 4.1 gives a simple way to understand the Kernel Trick. Imagine two concentric physical regimes: safe operating points occupy the inner disk, while failure-prone points form an outer ring. In the original (x_1, x_2) sensor plane, no straight line can separate the inner circle from the surrounding annulus. If we map each point into a radial coordinate such as $r^2 = x_1^2 + x_2^2$, the two classes become separated by a simple threshold in radial feature space. The SVM is still linear in the transformed representation, but that linear threshold appears as a circular decision boundary in the original coordinates. Kernel functions make this possible without explicitly building every transformed coordinate.

4.5 Engineering Context: HD-LSS Regimes

In advanced manufacturing, we frequently encounter the **High-Dimension, Low-Sample-Size (HD-LSS)** regime. Consider a production line where sensors sample $p = 500$ data points per unit, but because physical failures are rare, we only have $N = 30$ historical failures on record.

When $p > N$, standard regression models (like Logistic Regression) collapse because the design matrix is fundamentally underdetermined. Deep Neural Networks will violently overfit to the sparse noise.

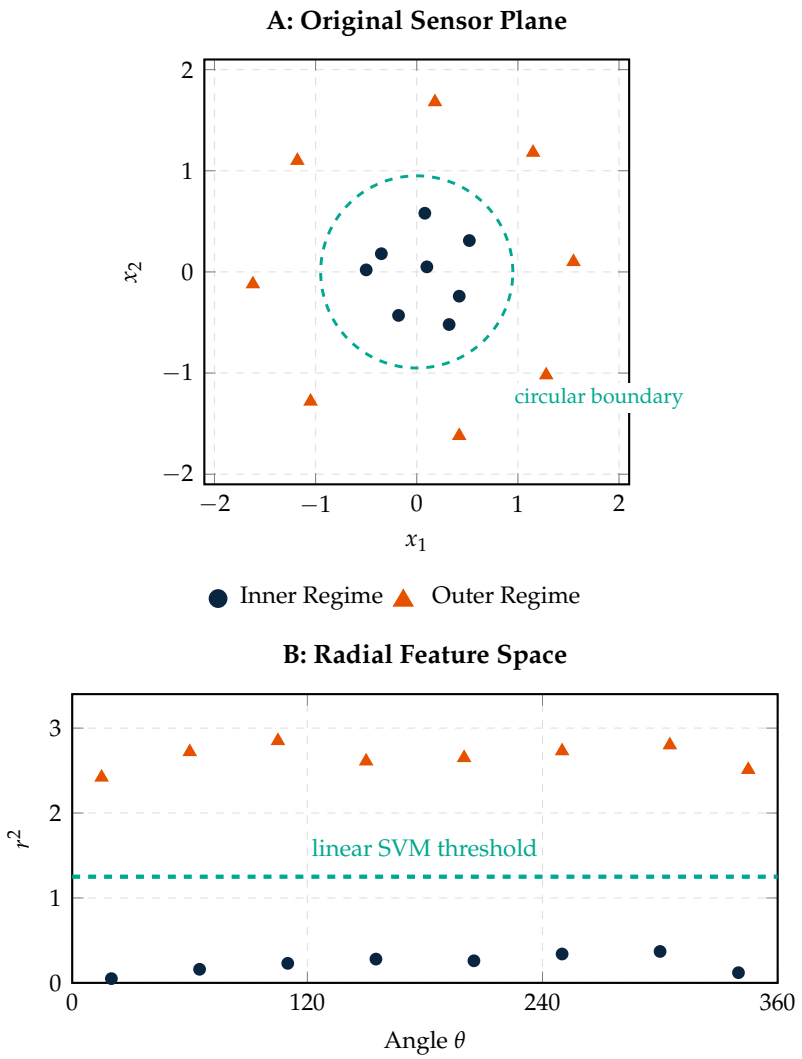


Figure 4.1: Kernel trick for concentric operating regimes. Panel A shows that inner and outer regimes cannot be separated by a straight line in the original sensor plane; the dashed circle is the nonlinear boundary. Panel B maps the same points to the radial feature $r^2 = x_1^2 + x_2^2$, where the two regimes are separated by a simple linear threshold.

SVMs survive the HD-LSS regime gracefully. Because the SVM objective function minimizes $\|w\|^2$ (a built-in geometric regularization) and relies strictly on the KKT complementary slackness (meaning it only cares about the Support Vectors, ignoring the bulk of the distribution), it can formulate robust decision boundaries even when the sensor dimensions vastly outnumber the physical samples.

4.6 Case Study: Pre-Failure Detection in Li-Ion Cells

Case Study: Battery Thermal Runaway Prediction

The Setup: An engineer must identify defective Lithium-Ion cells before they leave the factory. A defective cell is prone to thermal runaway (Class 1), while a healthy cell is stable (Class -1).
Cross-Disciplinary Reading: This is a rare-defect detection problem in manufacturing. The same logic applies to detecting cracks in mechanical parts, weak solder joints in electronics, defective medical devices, or early faults in rotating equipment: failures are rare, measurements are many, and the classifier must create a conservative safety boundary.
The Data Scarcity: During the "formation cycling" phase, the voltage curve is sampled 500 times ($p = 500$). The factory only has $N = 25$ known failure cases on record. This is a severe HD-LSS environment.
The ML Solution: The engineer deploys a Support Vector Machine that identifies the exact historical cells (the Support Vectors) that lie on the margin between stable and defective behavior. Any new battery cell whose high-dimensional voltage profile crosses the $w^T x + b = 0$ hyperplane is automatically flagged and isolated from the production line.

In Figure 4.2, the solid teal line represents the decision boundary. The dashed lines represent the physical safety margins. The circled data points are the **Support Vectors**, where $\alpha_i > 0$.

4.7 Summary

In this lesson, we established the mathematical tools to construct robust boundaries under extreme data scarcity:

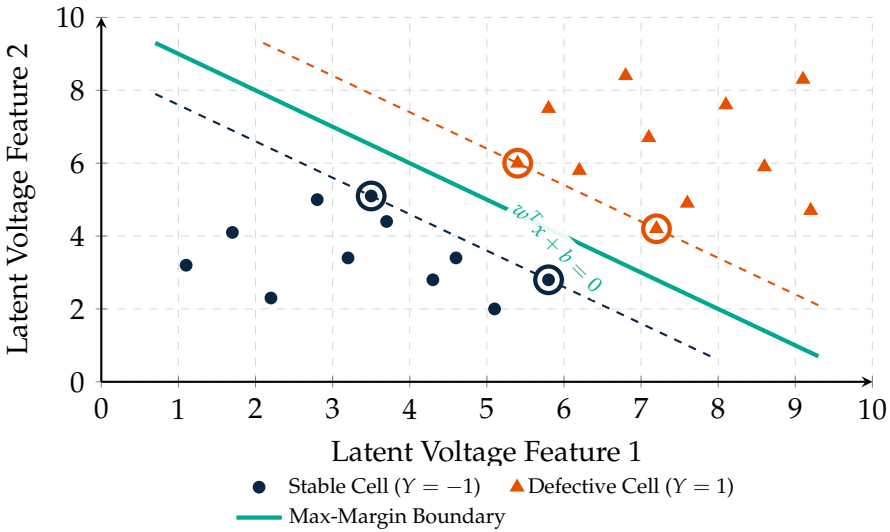


Figure 4.2: SVM max-margin boundary for latent battery-cell features. The teal decision boundary and the two dashed margin lines are rotated diagonally, making the geometric separation more natural while preserving the support-vector interpretation. Circled points lie on the active margin constraints.

- **Margin Maximization:** We defined the SVM objective (Equation 4.3) as minimizing $\|w\|^2$ subject to margin constraints, ensuring the safest possible boundary between physical states.
- **KKT Conditions & The Dual Problem:** We explicitly laid out the five KKT conditions and utilized Complementary Slackness (Equation 4.9) to prove that SVMs rely exclusively on a sparse subset of points.
- **Solving the SVM:** We established that the Dual Formulation (Equation 4.10) is solved via Quadratic Programming (QP), allowing us to reconstruct the physical boundary using Equations 4.5 and 4.12.
- **The Kernel Trick:** We bypassed computationally impossible feature mapping by using Kernel functions (Equation 4.14) to draw highly non-linear physical boundaries.
- **Engineering Application:** We proved that SVMs are optimal for

High-Dimension, Low-Sample-Size (HD-LSS) regimes, such as predicting battery failures using sparse manufacturing data.

References for Part I

1. Box, G. E. P. (1976). "Science and statistics." *Journal of the American Statistical Association*, 71(356), 791–799. *(Source for the Lesson 1 epigraph and the model-parsimony framing).*
2. Tukey, J. W. (1962). "The future of data analysis." *The Annals of Mathematical Statistics*, 33(1), 1–67. *(Source for the Lesson 2 epigraph and the right-question-versus-exact-answer principle).*
3. Einstein, A. (1934). "On the method of theoretical physics." *Philosophy of Science*, 1(2), 163–169. *(Published version of the 1933 Herbert Spencer Lecture and source for the Lesson 3 epigraph).*
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. *(Core reference for Bias-Variance, PCA, and SVM algorithms).*
5. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. *(Foundational reference for Maximum Likelihood Estimation and Logistic Regression).*
6. Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. *(The definitive text for Lagrangian duality, KKT conditions, and gradient-based solvers).*
7. Vapnik, V. N. (1998). *Statistical Learning Theory*. Wiley-Interscience. *(Primary source for Margin Maximization and Support Vector Machine theory).*
8. Cortes, C., & Vapnik, V. (1995). "Support-vector networks." *Machine Learning*, 20(3), 273–297. doi:10.1007/BF00994018. *(Original support-vector networks paper; supports the Lesson 4 derivation and terminology).*
9. Wold, S., Sjöstrom, M., & Eriksson, L. (2001). "PLS-regression: a basic tool of chemometrics." *Chemometrics and Intelligent Laboratory Systems*, 58(2), 109–130. *(Essential reading for Partial Least Squares in chemical engineering).*
10. MacGregor, J. F., & Kourti, T. (1995). "Statistical process control of multivariate processes." *Control Engineering Practice*, 3(3), 403-414. *(Application of PCA/PLS to latent variable tracking in plant sensor arrays).*
11. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. *(Reference for Gradient Descent optimization and the HD-LSS*

capacity dilemma).

12. Venkatasubramanian, V., Rengaswamy, R., Yin, K., & Kavuri, S. N. (2003). "A review of process fault detection and diagnosis: Part I: Quantitative model-based methods." *Computers & Chemical Engineering*, 27(3), 293-311.
13. Schweidtmann, A. M., et al. (2021). "Machine Learning in Chemical Engineering: A Perspective." *Chemie Ingenieur Technik*, 93(12), 2029-2039.